Summary of Two Papers on Hallucination Mitigation in Large Language Models

Objectives, Methods, Results, and Future Directions

Overview

This document summarizes two papers on hallucination mitigation in Large Language Models. Paper 1 is a 2025 survey that looks at the problem from a capability enhancement angle, covering RAG, reasoning, and Agentic Systems. Paper 2 is a 2026 Neural Networks article that introduces Emotion Augmented Inference (EAI), a method based on Plutchik's Wheel for affective alignment. For each paper, we cover Objectives, Methods, Results, and Future Directions, and then end with a comparison.

Aspect	**Paper 1: Survey on RAG, Reasoning, and Agentic Systems**	**Paper 2: Wheel of Emotions Based Hallucination Mitigation (EAI)**
Authors	Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, Mingming Liu (2025)	Zhenyu Wang, Jianmin Wang, Zenan Lu, Fang You (2026, Neural Networks)
Type	Survey paper that builds a taxonomy (arXiv 2510.24476v1)	New method paper with experiments
Main idea	Fix hallucinations by improving the model's knowledge access, reasoning, and planning	Fix hallucinations by aligning the model's emotional output with what the input actually shows
Output	A unified framework that maps each method to the type of hallucination it addresses	The EAI algorithm plus a 3 by 2 by 3 experimental design

Paper 1. A Survey on Hallucination Mitigation in LLMs Using RAG, Reasoning, and Agentic Systems

1.1 What the paper is trying to do

The paper studies hallucination in LLMs. Hallucination means content that sounds correct but is either factually wrong or does not match what the user asked for. The authors point out that this

happens at every model size, from very large models like ChatGPT 4 and DeepSeek R1 down to smaller ones like LLaMA 7B and Mistral. It is a serious issue in fields where errors cost real money or harm people, such as medicine, law, and scientific research.

Earlier surveys usually sort hallucinations by where they happen in the model lifecycle, like during pre training, fine tuning, or decoding. They also sort them by which part of the model is involved, like data, architecture, or output. This survey takes a different angle. It asks a simple question: which model capabilities, when made stronger, lead to fewer hallucinations? The authors focus on three approaches and explain why these three matter most:

- Retrieval Augmented Generation, or RAG.
- Reasoning enhancement.
- Agentic Systems.

They picked these three for four reasons. First, these approaches add capability instead of just blocking output. Second, they keep the model creative. Third, they are easy to deploy. Fourth, they are already used in real products like Grok 4, ChatGPT 4, and Gemini.

The paper's main contributions are:

- A clear review of the RAG pipeline, split into Precise Retrieval and Broad Retrieval.
- The first side by side comparison of three reasoning methods, namely Chain of Thought (CoT), Tool Augmented Reasoning, and Symbolic Reasoning, focused on hallucination.
- A formal definition of Agentic Systems and how they work as a unified framework.
- Real application examples in healthcare, law, finance, education, code generation, math, software development, and scientific research.

1.2 Two main types of hallucinations

- **Knowledge based hallucinations:** the facts are wrong but they look believable. Examples include citing papers that do not exist, getting historical dates wrong, or making up technical terms or institutions. You can check these against external databases or trusted sources.
- **Logic based hallucinations:** the facts may be correct but the reasoning is broken. Examples include drawing wrong conclusions from correct premises, circular reasoning, mixing up cause and effect, or generating code that does not follow logical structure.
- **Composite hallucinations:** a mix of both. These are common in multi step or multi tool workflows where one mistake leads to another.

1.3 RAG methods (for fixing knowledge based hallucinations)

The survey breaks RAG into three stages: pre retrieval, retrieval, and post retrieval. Each stage has its own techniques.

1.3.1 Pre retrieval stage (figuring out what the user wants)

- **Query rewrite:** Ma et al. rewrite the user's query so it works better for retrieval. RQ RAG trains the model to break down complex queries into simpler ones. Watson et al. show that many hallucinations actually start because the original query was unclear.
- **Auxiliary models:** SlimPLM by Tan et al. uses a small model to draft a quick answer first, then identifies which pieces of knowledge are missing and need to be retrieved.
- **Multi turn dialogue:** LARA looks at past messages in the conversation to better understand the current query.
- **Retrieval feedback (iterative):** RA ISF takes the result of one retrieval step and uses it to decide what to retrieve next, breaking the question into sub questions if needed. KiRAG writes a fresh query after each retrieval step based on the reasoning chain so far.

1.3.2 Retrieval stage (actually finding the documents)

Retriever types: There are four kinds. (i) Traditional sparse retrievers like BM25 and TF IDF use inverted indexes. They are fast and easy to interpret but bad at understanding meaning. (ii) Neural sparse retrievers like SPLADE and DeepImpact learn how important each term is and still output sparse representations. (iii) Dense bi encoders like DPR, Contriever, SBERT based, and ANCE map both queries and documents into the same vector space. (iv) Hybrid retrievers like ColBERT and ColBERTv2, plus BM25 combined with a dense reranker, mix lexical and semantic signals. Arivazhagan et al. found that hybrid retrievers reduce hallucinations more than purely sparse or purely dense ones. Real systems using hybrid include Blended RAG, CG RAG, and HyPA RAG.

Retrieval granularity: How big should each retrieved chunk be? Options range from full document, to chunk, to section, to sentence, to token, to entity. Coarse grained retrieval is faster but adds noise. Fine grained retrieval is precise but loses context. Chunk level is the most common balance. MoG and KET RAG use multiple granularities at once. KET RAG was specifically built to fix Graph RAG's weakness with entity level relations.

Reranking: This is needed because of the lost in the middle problem (Liu et al. 2024), where models pay less attention to information in the middle of the input. Approaches include feature based scoring (BM25, TF IDF), learning to rank (Pointwise, Pairwise like RankNet, Listwise like LambdaMART), neural rerankers based on BERT, LLM based rerankers like LLM4Ranking using GPT 4, PaLM, or LLaMA, and reinforcement learning based rerankers like Rank R1, which is task aware and good for multi hop QA.

Document preprocessing: TrustRAG masks out expressions that tend to cause hallucinations. LongLLMLingua compresses prompts in a question aware, coarse to fine way. DSLR pulls out the sentences relevant to the query and reorders them in their original order.

1.3.3 Post retrieval stage (using the retrieved info)

- **Knowledge integration strategies:** There are three. (i) Input level integration concatenates retrieved docs into the prompt. This is the most common. (ii) Intermediate layer integration (also called semi parametric) fuses through cross attention, adapters, or memory routing. This works best for factual accuracy. (iii) Output level integration adjusts the output distribution using rerank tokens, output filtering, or guided decoding. This is the most modular but the weakest at semantic alignment.
- **Knowledge conflicts:** RE RAG gives a confidence score to each retrieved document. C RAG uses contrastive explanations to push the model to think more carefully.
- **Post hoc checking:** compares the generated output with retrieved evidence at the meaning level to catch claims that are not actually grounded.
- **Traceability:** Explainable AI methods link each part of the output back to its source document so users can verify.

1.3.4 Precise Retrieval (for structured domains)

- **Graph Augmented RAG:** GraphRAG by Edge et al. builds a graph from documents and uses community detection to summarize hierarchically. CG RAG by Hu et al. uses citation graphs to reason across papers in scientific QA.
- **Knowledge Graph (KG) RAG:** works well in domains where knowledge is stable and structured, like medicine, law, and policy. DALK applies KG RAG to Alzheimer's QA. HyPA RAG uses legal entity and statute relations. GNN RAG uses graph neural networks to score nodes and extract paths. ToG (Think on Graph) lets the LLM actively walk reasoning paths on the graph.
- **Hybrid RAG:** HybridRAG runs KG retrieval and vector retrieval at the same time and combines the results. Think on Graph 2.0 alternates between them iteratively. NodeRAG builds one heterogeneous semantic graph that holds both entity and document nodes for unified retrieval.

1.3.5 Broad Retrieval (for the open web)

- **Cross domain generalization through web search:** WebGPT and IALM are early systems. WebGLM improved efficiency. Toolformer learns when to call a search engine. WebWalker uses a multi agent setup for deep web navigation. DeepResearcher uses reinforcement learning for end to end retrieval planning. FoRAG uses dual granularity RLHF to improve factuality in long form QA. HtmlRAG keeps the original HTML structure during preprocessing.

- **Multi modal RAG:** MuRAG was first to extend retrieval to text plus image. VisRAG embeds images directly. HtmlRAG retrieves from raw HTML. The downside is that lining up different modalities can introduce new hallucinations from cross modal ambiguity.
- **Long context comprehension:** approaches include sparse attention (Native Sparse Attention, XAttention), linear attention, chunk based recursive processing (GraphReader, ReadAgent), and fine tuning. Iterative chunk based processing keeps short context sensitivity intact and works well with reranking.
- **AI generated content (AIGC) identification:** Watermarking is the most effective method but breaks if the text is paraphrased or translated. GLTR and DetectGPT use statistical features like lexical diversity and perplexity. RoBERTa based classifiers are used for fake news. Shumailov et al. warn that AIGC contamination can cause model collapse over time.

1.4 Reasoning enhancement methods (for fixing logic based hallucinations)

1.4.1 Chain of Thought (CoT)

Few shot CoT (Wei et al. 2022) shows the model examples of step by step reasoning. Zero shot CoT (Kojima et al.) showed that simple prompts like 'Let's think step by step' or 'Wait' (Muennighoff et al.) can activate reasoning. Natural Program (Ling et al.) splits CoT into sub processes and verifies each step. SymbCoT (Xu et al.) translates the problem into symbolic form, applies logic rules, and verifies in order. Self Consistency (Wang et al.) generates multiple CoT paths and picks the answer by majority vote. Reasoning path supervision (Zhu et al.) trains the model to learn complete coherent chains in long context tasks. Adaptive variants like AdaCoT, Dynamic CoT, and D CoT let the model decide when to use CoT and how long the chain should be.

There is a catch. Liu et al. show that CoT can actually hurt performance on tasks where humans also do worse when they overthink.

1.4.2 Tool Augmented Reasoning

Instead of doing everything in weights, the model calls external tools. These include calculators, code interpreters (Program of Thoughts, or PoT), retrieval engines, and symbolic engines. ReAct (Yao et al.) and Toolformer (Schick et al.) are the foundational works. TART handles table QA. ChatCoT does tool augmented CoT on chat models. Dynamic tool use frameworks like ToolFive, ANSWERED, and SciAgent let the model decide whether to use a tool, when to use it, and which one to use. Reflective systems re check their outputs and reconfigure tool use as needed.

1.4.3 Symbolic Reasoning

The LLM acts as a controller that translates natural language into symbolic logic. A logic engine then performs deterministic deduction, and the result is translated back into natural language. ChatLogic (Wang et al.) does exactly this: the LLM converts to logic, the engine reasons, then translates back. It is highly reliable but only works for narrow types of problems. Logic LM uses an LLM for the translation step then calls a symbolic reasoner. SymbCoT does CoT in symbolic space and adds a verifier. This is the broader neuro symbolic AI direction, which combines LLM language understanding with verifiable symbolic logic, useful in high stakes domains.

1.4.4 Reasoning applications

- **Code generation:** Foundation models include StarCoder, CodeGen, and Code Llama. CoT based improvements include Yang et al. applying CoT to small models, SCoTs by Li et al. generating structured reasoning aligned with program logic, and Zheng et al. using AST guided coarse to fine generation.
- **Mathematical reasoning:** the well known 'is 9.11 greater than 9.9?' failure shows the core weakness. Symbolic approaches include Gaur et al. self prompting for symbolically valid paths and Dhanraj et al. mapping hidden states to a symbolic vector space. Tool based approaches include PoT (Chen et al.), MathSensei (Das et al.), and SciAgent (Ma et al.).

1.4.5 The overthinking problem

Cuadron et al. and Chen et al. show that deeper reasoning is not always better. Models can get stuck on irrelevant paths and produce redundant or hallucinated content. The solution is dynamic depth regulation, where the model adjusts how long it thinks based on the task.

1.5 Agentic Systems (for fixing composite hallucinations)

The authors define an Agentic System as an LLM that has at least reasoning capabilities and retrieval modules. The reasoning is simple. RAG alone fixes facts but not logic. Reasoning alone fixes logic but lacks grounding. Composite hallucinations need both.

- **Agentic Reasoning by Wu et al.:** uses a Mind Map Agent to build a structured reasoning graph and a Web Search Agent for real time retrieval. The authors call this the basic agentic form.
- **MA RAG by Nguyen et al.:** a multi agent setup with a planner, step definer, evidence extractor, and QA agent, supporting dynamic plan retrieve synthesize cycles.
- **HM RAG:** three layer architecture for task decomposition, modality aware retrieval, and decision fusion. Handles multi modal composite hallucinations.
- **SWE agent:** for software engineering. Uses an Agent Computer Interface for structured retrieval over codebases plus feedback driven iterative reasoning.

- **AI Scientist v2:** for scientific research. Uses Agentic Tree Search over hypothesis spaces, with multi branch reasoning and verification cycles.

1.6 Benchmarks

For knowledge based hallucinations: TruthfulQA (817 examples, intrinsic knowledge), MedHallu (10,000 medical, RAG focused), RAGTruth (18,000), FreshQA (recency), and HalluLens (splits into intrinsic and extrinsic).

For logic based hallucinations: BIG bench (200, outcome correctness), PrOntoQA (40,000, intermediate proof verification), ProofWriter, ToolBench (16,464, tool use), API Bank, LogicBench (symbolic reasoning), and ReClor and LogiQA (LSAT and GMAT style argument validity).

For composite hallucinations: AgentBench (17,000, eight environments covering OS, databases, web, games, knowledge reasoning, and puzzles), L MARS (200, legal multi turn QA), InfoDeepSeek (245, web retrieval), and R Judge (safety risks).

1.7 Findings and discussion

- **RAG complexity paradox:** retrieval failures or unnecessary retrievals can themselves create new hallucinations. The authors call this 'mitigating hallucinations with hallucinations.' Adaptive retrieval is the central direction for fixing this.
- **CoT limits:** CoT is purely linguistic and does not have verifiable logical grounding. Turpin et al. showed that 'language models don't always say what they think.' CoT also tends toward overthinking. The way forward is integrating CoT with symbolic and tool augmented reasoning.
- **Agentic challenges:** there is no standard for retrieval reasoning integration. Most systems run sequential rather than synergistic pipelines, errors propagate down the chain, and computational cost is high.

1.8 Future directions

- Adaptive retrieval strategies that model intent accurately and handle retrieval failures gracefully.
- Source reliability assessment that quantifies credibility before integration.
- Long text and multi source fusion with noise reduction across domains and modalities.
- Generalized tool integration frameworks that adaptively pick tools based on task uncertainty.
- Co evolution of retrieval and reasoning at the architectural level, with closed loop self reflection (plan, retrieve, reason, verify).

- Explainability mechanisms that trace how external knowledge is used in reasoning.
- Lightweight modules that balance accuracy and latency.
- Adaptive control that balances factuality and creativity.
- Cross modal consistency checking for multi modal hallucinations.
- End to end, process level agentic benchmarks that jointly assess knowledge and logic hallucinations across modalities.

Paper 2. Hallucination Mitigation Based on the Wheel of Emotions

2.1 What the paper is trying to do

This paper targets hallucination in multimodal generation, where the output looks coherent but does not match the facts or context. The authors argue that earlier approaches like data augmentation, contextual alignment, dataset fine tuning, post processing correction, and RLHF miss the affective and cognitive mechanisms behind generation. They specifically point out that in conversational systems, when the model's emotional state is unstable, hallucinations get worse.

Their proposed method is called Emotion Augmented Inference (EAI). It is built on Plutchik's Wheel of Emotions, a psychological model. EAI is supposed to perceive, regulate, and reconstruct emotional signals during generation. The contributions are:

- The EAI framework, which combines visual and textual emotional signals so that the model generates emotionally consistent output.
- A triadic emotional alignment mechanism connecting dialogue context, target emotion, and generation preference, with logits based dynamic sampling.
- A multi context emotional evaluation protocol that tests hallucination under three emotional contexts: neutral (S1), positive (S2), and negative (S3).

2.2 The theoretical basis: Plutchik's Wheel of Emotions

Plutchik's wheel models emotions as a hierarchical, symmetrical structure. There are basic emotions like joy, sadness, anger, and fear, and composite emotions like optimism, shame, and relief, formed by combining basic emotions and varying their intensity. EAI uses the wheel as a coordinate system. The model's emotional logit distribution is mapped onto the wheel's spatial coordinates, which lets the system align and intervene on emotions.

The wheel plays three roles inside EAI:

- **Emotion visualization:** emotion logits are converted into a 2D polar coordinate emotion distribution map, which makes it easy to track where emotions are drifting.
- **Emotion hallucination suppression:** Contrastive Decoding gives lower confidence to emotion categories that conflict with the input. The penalty is based on angular differences in Plutchik's eight dimensional space. For example, if the image shows loneliness but the model generates excitement, the system applies a penalty.
- **Emotion regulation:** Emotional Boosting Factors (EBFs) dynamically adjust how much positive, negative, and neutral emotion appears in the output. They reinforce the desired emotion, balance multiple emotional states, and suppress hallucinated emotions.

2.3 The EAI architecture

EAI runs in three steps.

Step 1. Multimodal emotional input modeling. Inputs are text t, normal visual input v (the realistic emotional context), and distorted visual input v prime (an emotionally misleading version). A vision language function f computes two raw emotion logits:

logits(y | t, v) = f(t, v)

logits(y | t, v prime) = f(t, v prime)

Then contrast decoding with suppression coefficient alpha gives:

L(y) = (1 + alpha) * logits(y | t, v) minus alpha * logits(y | t, v prime)

This pushes the model toward emotion that comes from the realistic input and away from the distorted one.

Step 2. Emotional contrast mapping and guidance. A sentiment bootstrapping term is added on top:

L prime (y) = L(y) + beta * E(y)

Here E(y) is +1 for positive emotions, 0 for neutral, and minus 1 for negative. Beta is the alignment factor. Setting beta greater than zero amplifies positive emotions; setting beta less than zero suppresses negative ones. The adjusted logits are softmax normalized into a probability distribution P(y), and each y is mapped to a position on Plutchik's wheel with both semantic and visual meaning.

Step 3. Hallucination elimination and final optimization. A hallucination indicator and penalty are applied:

I_hall(y) = 1 if y belongs to HallucinatoryEmotions, otherwise 0

F(y) = sigmoid(L prime (y) minus gamma * I_hall(y))

Here gamma is the hallucination suppression factor.

Detection by embedding distance:

$$I_{hall}(y) = 1 \text{ if the L2 norm of } (E(y) - E^*(x)) \text{ is greater than delta, otherwise } 0$$

E(y) is the emotion embedding of the generated text, E*(x) is the true emotion of the input modality, and delta is the tolerance threshold.

Multimodal fusion of input emotions, with embeddings extracted via CLIP, is:

$$E^*(x) = \lambda * E_{text}(x_t) + (1 - \lambda) * E_{image}(x_v)$$

Lambda is between 0 and 1 and is set based on which modality is more confident.

EBF normalization uses a sigmoid:

$$EBF_i = \text{sigmoid}(\eta_i) = 1 / (1 + \exp(-\eta_i))$$

Eta_i is the internal weight for emotion category i, and EBF_i is the resulting boost or suppression factor on per token logits.

2.4 Experimental setup

Hardware: NVIDIA RTX A5000 with 24 GB VRAM and mixed precision support, 32 GB DDR4 RAM, 2 TB NVMe SSD, Windows 10 x64, and CUDA 11.8.

Software: PyTorch 2.0, Hugging Face Transformers, NumPy, Pandas, and Matplotlib.

Datasets: MSCOCO with about 330,000 images (200,000 human annotated, 80 categories, 5 captions each, plus instance segmentation) and GQA with 113,000 images and over 22 million QA pairs based on scene graphs.

Sampling strategies: Random, Weighted Random, and Most Common. The third is the hardest because it uses common distractors with high co occurrence.

Baselines: LLaVA 1.5 (CLIP based vision language alignment), Qwen VL (Alibaba DAMO), and InstructBLIP (instruction fine tuning). Each was tested under Regular decoding versus EAI decoding.

Design: 3 baseline models by 2 datasets by 3 sampling strategies. Metrics are Accuracy, Precision, Recall, and F1.

Affective metrics: CLIP for vision text emotion features, and VADER for emotion scoring on the expression image y.

2.5 Results

2.5.1 Quantitative results on multimodal QA and image captioning

Across all setups, EAI improves baselines by 4 to 8 percent on Accuracy, Precision, Recall, and F1. Some examples:

- **MSCOCO, Random, LLaVA 1.5:** Accuracy goes from 81.06% to 89.33%; F1 from 80.68% to 88.87%.
- **MSCOCO, Random, Qwen VL:** Accuracy from 86.66% to 88.41%; F1 from 84.20% to 89.93%.
- **MSCOCO, Random, InstructBLIP:** Accuracy from 81.14% to 84.78%; F1 from 78.11% to 87.72%.
- **GQA, Random, LLaVA 1.5:** Accuracy from 73.41% to 78.27%; F1 from 74.41% to 79.26%.
- **GQA, Random, Qwen VL:** Accuracy from 74.96% to 79.83%; F1 from 74.44% to 80.44%.

EAI also has a more concentrated distribution than Regular in the box plots, which means it is more stable. The strategy is robust under Random and Weighted Random sampling and still adds gains under Most Common, the hardest setting.

2.5.2 Three emotional contexts (S1, S2, S3)

Twenty master's and doctoral students evaluated five ChatGPT versions (4o, o1, o1 mini, 4o mini, and 4) under three emotional contexts: neutral S1, positive S2, and negative S3. They used Accuracy and a Likert emotion management score.

- **S3 (negative) outperforms S1 by 5.48% on accuracy and 35.98% on emotion management. It outperforms S2 by 2.23% on accuracy and 15.15% on emotion management.**
- **4o accuracy across contexts:** 88.27% (S1), 91.07% (S2), and 93.10% (S3). Emotion management peaks at 95.96 in S3.
- **Lightweight versions** (o1 mini and 4o mini) have slightly lower accuracy but smaller standard error, which means more stability for resource constrained deployment.

2.5.3 Image generation case studies

- **Robot in lab:** Regular decoding produces unfounded subjective terms like 'depressed,' 'melancholic,' and 'dim environment.' EAI keeps the output to perceptible neutral expressions like 'contemplative' and 'calm expressions.'
- **Sunset valley with wildflowers:** Regular labels with romanticized terms like 'romantic,' 'sentimental,' and 'gently releasing emotions.' EAI gives 'restrained melancholy' and 'contemplative emotions,' which is more objective.
- **Mountain lake meadow:** Regular generates positive hallucination labels like 'vitality' and 'joy.' EAI produces 'a natural scene, evoking hope and joy,' which is grounded in lighting and scene elements.

2.5.4 Image transparency robustness

The transparency levels tested were 0, 15, 30, 50, 65, 80, and 95 percent.

- **Transparency at or below 50%:** Accuracy and F1 stay above 85%, which is strong robustness.
- **Transparency at or above 65%:** Performance drops sharply. At 95%, both Accuracy and F1 fall to around 30%.
- **Time cost:** rises rather than falls under high transparency. This shows a 'high computational cost, low recognition' imbalance for degraded inputs.

2.5.5 Cross LLM comparison: EAI vs ChatGPT 4o vs ERNIE Bot V3.3.0

There were 40 questions in total: 20 Chinese and 20 English; 10 history, 6 politics, and 24 economics. Each question was repeated three times. Scoring was 5 points for correct, 2 points for 'I don't know,' and 0 points for incorrect.

- EAI outperforms both baselines across most categories and languages, with the largest margins on English history and politics.
- On economics, the gaps narrow and ChatGPT slightly leads in resistance to hallucinations.
- ERNIE Bot's 'I don't know' strategy reduces nonsense generation but trades off coverage.
- Overall, EAI is 100 to 125 percent better than ERNIE Bot and 5 to 33 percent better than ChatGPT in these tests.

2.6 Limitations

- Limited emotional coverage. The work focuses on common emotions. Rare or complex emotions like guilt, envy, and mixed states are underexplored due to insufficient annotated data.
- No audio modality. Tone and pitch are not yet integrated.
- Long form coherence. Multi turn dialogue consistency is not validated. Reinforcement learning or memory augmented architectures may be needed.
- Computational cost. Visual emotion contrast decoding and emotion text symbolization are expensive, which limits deployment on mobile or embedded devices.
- Demographic generalizability. The studies were run in controlled settings with limited participant diversity. User centered studies are needed.
- Cross lingual generalization, redundancy, and readability still need work.

2.7 Future directions

- Expand the emotional dataset to cover rare and composite affective states.
- Add audio modality and design vision language audio fusion for richer affective grounding.
- Combine EAI with reinforcement learning to dynamically optimize affective output, especially in multi turn dialogue.
- Build lightweight EAI variants for real time and resource constrained deployment.
- Run user centered evaluations in healthcare, education, and customer service to assess trust and satisfaction.
- Improve cross lingual generalization and interpretability.